

SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Shen Ivin Clement	Reg. No.:	192371023
Section / Batch:	2023-2027	Date:	26-08-2026

Instructions to Students

• Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols. • Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities. • Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration. • Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time. • Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu s$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions with	Develops workable solutions with	Solutions partially	Solutions do not meet constraints

CO5 Statement. Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe and Thunderbolt.

*Every answer follows the rubric flow: **restate constraints** → **analyse & prioritise them** → **develop a solution that meets them** → **apply I/O concepts** → **justify with computed performance measures**. Latency/bandwidth figures are representative and vary by device and generation.*

Constraints to satisfy

- CPU utilisation must stay below 15 %
- Multiple sensors raise interrupts every 1 ms
- Use vectored interrupt handling
- Choose memory-mapped vs I/O-mapped I/O for optimal latency

Constraint analysis & prioritisation

The binding constraint is **CPU utilisation < 15 %** under a fixed 1 ms interrupt cadence; everything else must serve that budget. Because utilisation = interrupt-rate × service-time, the levers are (a) minimise per-interrupt cost and (b) minimise per-interrupt data handling.

Solution — I/O mechanism

- **Vectored interrupts:** each sensor supplies its ISR address directly, so the CPU jumps straight to the handler with

no polling to identify the source → shortest dispatch and deterministic latency. • **Memory-mapped I/O (chosen):** device registers sit in the normal address space, so the CPU uses ordinary load/store instructions (full addressing modes, no special IN/OUT) → faster, lower-latency register access than I/O-mapped, which needs dedicated I/O instructions and a separate space. • **Short ISRs + DMA for any bulk data:** keep handlers to a few instructions (latch the sample, clear the flag) and defer heavier work; route any block data through DMA so the CPU is not copying bytes.

CPU-utilisation check ($U = \text{interrupt_rate} \times \text{service_time}$)

Assume 4 sensors, each interrupting every 1 ms → rate = $4 \times 1000 = 4000$ interrupts/s.

Per-interrupt cost (dispatch + save/restore + short ISR) $\approx 20 \mu\text{s}$.

$U = 4000 \times 20 \mu\text{s} = 0.08 = 8\% < 15\% \checkmark$

Head-room: budget allows $0.15 / 4000 = 37.5 \mu\text{s}$ per ISR, or up to 7 sensors at $20 \mu\text{s}$ (14 %).

Justification & constraint check

Vectored + memory-mapped gives the lowest, most deterministic latency; short ISRs plus DMA keep the CPU out of data movement.

Computed utilisation (8 %) sits comfortably under the 15 % cap with room to add sensors — all four constraints are satisfied.

Constraints to satisfy

- Storage throughput must exceed 3 GB/s
- Must use NVMe over PCIe
- CPU must not handle bulk data transfer
- Integrate a DMA controller with interrupt notification

Constraint analysis & prioritisation

Two hard constraints drive the design: a **> 3 GB/s** throughput floor and **no CPU involvement in bulk movement**. The first fixes the PCIe link width/generation; the second mandates DMA (bus-mastering) with completion interrupts rather than programmed I/O.

Solution — subsystem design

• **Link sizing:** a PCIe **Gen4 ×4** link (~ 7.9 GB/s raw) clears 3 GB/s with wide margin even after protocol overhead; Gen3 ×4 (~ 3.9 GB/s raw, ~ 3.4 GB/s usable) is borderline, so Gen4 ×4 is recommended. • **NVMe + DMA:** the NVMe controller is a PCIe bus master. Using submission/completion queues it **DMAs data directly to/from RAM**; the CPU only posts descriptors — it never copies payload. • **Interrupt notification with coalescing:** completions are signalled via **MSI-X**; interrupt coalescing batches many completions per interrupt, keeping CPU overhead negligible.

Throughput & CPU-cost check

PCIe Gen4 lane = $16 \text{ GT/s} \times 128\text{b}/130\text{b} \approx 1.97 \text{ GB/s/lane} \rightarrow \times 4 \approx \mathbf{7.88 \text{ GB/s}} > 3 \text{ GB/s} \checkmark$

At 3 GB/s in 128 KB DMA transfers → $3\text{e}9 / 131072 \approx 22,888$ transfers/s.

With coalescing at 1 interrupt per 64 transfers → ≈ 358 interrupts/s; ISR $5 \mu\text{s} \rightarrow$

$U = 358 \times 5 \mu\text{s} \approx 0.0018 = \mathbf{0.18\% \text{ CPU}} \rightarrow$ bulk transfer is effectively CPU-free $\checkmark$

Time to move 30 GB at 3 GB/s = **10 s** (link-bound, not CPU-bound).

Justification & constraint check

Gen4 ×4 NVMe with bus-master DMA and MSI-X coalescing exceeds 3 GB/s while using < 0.2 % CPU.
The CPU handles only queue setup and rare coalesced completions — satisfying all four constraints.

Constraints to satisfy

- 6 peripherals on USB, 2 high-speed devices on Thunderbolt
- Avoid interrupt starvation
- Use both hardware and software interrupts
- Ensure fair bus arbitration

Constraint analysis & prioritisation

The priority is **fairness** — no device may be starved of interrupt service or bus access — while still giving the two Thunderbolt devices their high bandwidth. This calls for a priority-with-fairness interrupt scheme plus a rotating bus arbiter.

Solution — architecture

- **Topology:** the 6 USB peripherals attach through an xHCI host controller; the 2 Thunderbolt devices (PCIe tunnelling, 40 Gbps ≈ 5 GB/s each) attach on dedicated PCIe lanes so they never contend with the USB tree.
- **Hardware + software (split) interrupts:** the hardware interrupt does the minimum (acknowledge, grab data), then schedules a **software interrupt / deferred routine** (softirq/DPC-style) for the heavier processing → ISRs stay short so no device is blocked.
- **Anti-starvation:** use a priority interrupt controller with **rotating (round-robin) priority and aging** among equal-priority devices, guaranteeing each source is serviced within a bounded time.
- **Fair bus arbitration:** a **round-robin arbiter** for the shared USB bus; Thunderbolt on separate lanes with its own arbitration → bounded worst-case wait for every device.

Bandwidth & fairness sanity check

Thunderbolt: $2 \times 40 \text{ Gbps} = 80 \text{ Gbps} = 10 \text{ GB/s}$ aggregate → needs ≥ PCIe Gen3 ×8 upstream headroom.

Round-robin among 8 interrupt sources: worst-case wait ≤ $7 \times (\text{max ISR time})$.

If each top-half ISR ≤ 3 μs → worst-case interrupt wait ≤ 21 μs, bounded and starvation-free ✓

Justification & constraint check

Short hardware ISRs + deferred software interrupts keep every handler responsive; rotating priority with aging removes starvation.

Round-robin arbitration bounds each device's bus wait, and separating Thunderbolt onto dedicated lanes preserves its bandwidth — all constraints met.

Constraints to satisfy

- Continuous data stream at 500 MB/s
- Minimal CPU intervention
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

Constraint analysis & prioritisation

A sustained **500 MB/s** stream with **minimal CPU intervention** makes the transfer method the decisive choice. The comparison below shows interrupt-driven I/O is infeasible at this rate, so **DMA** is mandatory; memory-mapped registers give fast control-path access.

Comparison — DMA vs interrupt-driven I/O

Who moves data	DMA controller → RAM directly CPU, one transfer per interrupt
Interrupts	1 per block (thousands/s) 1 per word/small unit (millions/s)
CPU load at 500 MB/s	A few % > 100 % (impossible)
Best for	High-rate streaming Low-rate / sporadic devices

CPU-utilisation comparison at 500 MB/s

DMA, 64 KB blocks: $500\text{e}6 / 65536 \approx 7,629$ blocks/s → 7,629 interrupts/s; ISR $5\text{ }\mu\text{s}$ → $U = 7,629 \times 5\text{ }\mu\text{s} \approx 0.038 = \mathbf{3.8\%}$ CPU ✓ (minimal intervention)

Interrupt-driven, 512 B/interrupt: $500\text{e}6 / 512 \approx 976,562$ interrupts/s; ISR $2\text{ }\mu\text{s}$ → $U = 976,562 \times 2\text{ }\mu\text{s} \approx 1.95 = \mathbf{195\%}$ → the CPU cannot keep up ✗

Solution

- **DMA (burst mode)** streams samples straight into a ring buffer in memory; the CPU is interrupted only once per completed block.
- **Memory-mapped I/O** for device registers (start/stop, buffer pointers, status) → fast load/store control with no special I/O instructions.
- **Double-buffering / ring buffer:** DMA fills one buffer while the CPU processes the previous → no data loss on the continuous stream.

Justification & constraint check

The calculation is decisive: interrupt-driven I/O needs ~195 % CPU and fails, while DMA needs only ~3.8 %. DMA + memory-mapped registers + double-buffering sustains 500 MB/s with minimal CPU intervention — all constraints satisfied.

Constraints to satisfy

- Support nested interrupts
- Maximum interrupt latency < $10\text{ }\mu\text{s}$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Constraint analysis & prioritisation

The hard real-time limit is **worst-case latency < $10\text{ }\mu\text{s}$** for high-priority devices, which only holds if higher-priority interrupts can **preempt** lower ones (nesting) and dispatch is fast (vectored). Background work must never block a high-priority source.

Solution — mechanism

- **Priority-based nested vectored controller** (NVIC-style): every source has a priority; a higher-priority interrupt **preempts** a running lower-priority ISR, so critical devices are not made to wait.
- **Vectored dispatch for high-priority devices**: the controller supplies the ISR address directly → no source-polling, minimal, deterministic entry time.
- **Software interrupts for background tasks**: deferred/low-priority work runs via a software-triggered interrupt (PendSV-style) only when no hardware interrupt is pending → background load never delays foreground events.
- **Keep interrupt-disabled critical sections short** so the worst-case blocking term stays small.

Worst-case latency budget (must be < 10 μ s)

Latency = finish-current-instruction + interrupts-disabled window + context save + vector fetch ≈ 0.2 μ s (instruction) + ≤ 2 μ s (critical section) + ~ 1 μ s (save 8–16 regs) + ~ 0.1 μ s (vector) Worst case $\approx$

3.3 μ s < 10 μ s ✓

Nesting ensures a high-priority request is not queued behind a lower-priority ISR, preserving the bound.

Justification & constraint check

Nested preemption + vectored dispatch bound the high-priority response to ~ 3.3 μ s, well inside the 10 μ s limit. Routing background work to software interrupts keeps the foreground path clear — every constraint is met with margin.